

Cheap Queries, Fixed Indexes: The Costs and Limits of Query Encoder Distillation

Dylan Couzon (Qdrant)

October 2026

Abstract

When is query-side distillation a practical way to keep an index and make queries cheaper? We measure build and serving costs for a frozen Stella index, and separately test teacher choice before indexing under one closed-form table recipe. Across 26 checkpoints (ten registered), teacher retrieval quality weakly ranks table quality (Spearman +0.09), while directly evaluating each table on development queries ranks it much better (0.88). The strongest registered teacher produces the weakest registered table. The development screen chooses Stella: its closed-form table scores 0.3974, against 0.2455 for the strongest teacher’s table, over their respective indexes on the same six datasets. This retrospective comparison is workload-dependent and does not validate the screen for other student recipes. Over the frozen Stella index, a separately trained table and a 34.5M transformer retain 81.4% and 90.5% of teacher nDCG@10 on BEIR-15. The transformer’s training loop takes 57.3 A100 hours, priced at about \$95 at the recorded rental rate, excluding teacher-target preparation and research costs. Serving adds another constraint: the table needs a wider HNSW search, reducing its roughly 60-fold encoding advantage over the transformer to 2.2-3.1-fold end to end on a one-million-passage diagnostic. The experiments distinguish the cost to fit a cheap encoder from the quality and search cost of using it.

1. Introduction

An organization has already encoded its documents with a strong embedding model. It wants cheaper queries, perhaps because queries run on a laptop CPU or because every request now pays for a large transformer. Replacing the whole retrieval model would also replace the document index. Query-side distillation offers another choice: train a smaller encoder to produce vectors in the existing model’s space, then search the same document vectors.

This construction is established in prior work [QED, EmbedDistill, pyNIFE, LEAF]. The practical question is what makes it worth doing. An inexpensive training loop is useful only if the resulting encoder retrieves well, and an impressive encoding speedup is useful only if it survives the search. We investigate **how teacher choice, preparation and training cost, and approximate search constrain cheap queries over a frozen index**.

The sharpest result comes from teacher choice. Under the same closed-form table recipe, gte-large-en-v1.5 scores 0.5970 as a teacher but yields a table scoring 0.2455. Stella, derived from that backbone, scores slightly lower as a teacher, 0.5745, but yields a much stronger table, 0.3974. A developer choosing by teacher quality would miss this reversal. Directly scoring the candidate tables on two development forums ranks their performance on six other datasets much better. This is evidence about a particular table recipe, not a general method for choosing distillation teachers.

The other two findings concern the cost boundary. Our compact transformer trains in 57.3 hours on one A100 with cached teacher targets; that training loop is about \$95 at the historical rental rate. Data preparation, target encoding, and recipe development add work. At serving time, our lookup table encodes a short query about 60 times faster than the transformer, but its vectors incur a larger approximate-search penalty.

Retuning HNSW narrows the end-to-end gap to about two to three times on the larger collection we measured.

These experiments inform two different decisions. If the document index already exists, its teacher is fixed: measure whether a cheap query path is viable for that space. If the index has yet to be built and a cheap query path matters, evaluate candidate cheap encoders before selecting its teacher. In both cases, evaluate the complete retrieval path on representative queries. The contributions are the cross-teacher table comparison, explicit component-cost accounting, and the same-index measurement of the search penalty. Supporting routing and query-perturbation experiments appear in the appendices.

2. Related work and contribution

Distilling queries while keeping documents fixed. QED trains a small query encoder by embedding alignment to a frozen document model [QED]. EmbedDistill studies geometric distillation, including asymmetric students [EmbedDistill]. LEAF explicitly targets modest data and infrastructure for teacher-aligned encoders [LEAF], and NanoVDR distills the text query path of a visual-document retriever [NanoVDR]. These works establish that query-side training can be practical; approachability alone is not this paper’s novelty.

Replacing the query network with a table. Model2Vec and static sentence embeddings average token vectors [Model2Vec, StaticEmb]. pyNIFE fits a lookup table against a frozen off-the-shelf teacher, reuses its document index, and proposes switching between a static and a contextual query path [pyNIFE]. LightRetriever trains a lookup-table query side jointly with an LLM document side [LightRetriever]. Our table architecture follows this prior work. We hold the document side fixed, compare one table recipe across many checkpoints, and measure its behavior inside an approximate-search engine.

Teacher choice and retrieval decisions. A stronger teacher need not make a stronger student in knowledge distillation [Cho and Hariharan 2019], including dense retrieval [PROD]. NanoVDR finds teacher quality predictive of student retention across datasets for one fixed teacher [NanoVDR]; our study varies teachers under one table recipe on the same datasets. Those are different comparisons. Retrieval strategy selection and query performance prediction also have prior work [Arabzadeh et al., QPP]. Compatible representation learning addresses index reuse when retrieval models change [BCT, FCT]. Our routing experiments are a bounded study of two compatible query encoders, not a claim to invent adaptive retrieval.

The empirical contribution is to connect three measurements that an encoding benchmark alone leaves apart: which cheap query representation a teacher supports, what fitting it costs, and what it costs to search with it. We do not claim a new distillation architecture, a universal teacher-selection rule, or a generally superior small model.

3. What has to be built, and what does it cost?

3.1 One document space, several query encoders

The document encoder is NovaSearch/stella_en_400M_v5 at revision ffeb2b7e: normalized 1024-dimensional vectors, fp32 encoding, fp16 storage, cosine similarity. A replacement query encoder must align with that space. Matching vector width alone is insufficient; prompts, pooling, tokenization, normalization, and the training targets matter. The index is kept fixed for the served-tier and deployment comparisons.

Query path	Computation per query	Construction
Stella	the original 400M transformer with its query prompt	no query student to build
Nano	a 34,540,672-parameter transformer	bge-small layers 12, 8, and 4, a linear head to 1024 dimensions, mean pooling; squared L2 alignment to Stella targets
Zero	token lookup, count-saturated pooling, normalization	30,522 token rows trained with cosine and ranking losses, then exported as int8
Closed-form probe	token lookup, pooling, normalization	ridge regression from token counts to a teacher’s query vectors, anchored at teacher-derived token rows

Zero and the closed-form probe are different recipes. Section 4 studies the probe across teachers; the released Zero and Nano recipes were trained against Stella only. A good probe result therefore supports that closed-form representation, without establishing the result of training another student.

3.2 Preparation is separate from fitting

A query-only alignment objective can learn from query text and cached teacher query vectors. Ranking losses also need the appropriate document vectors or training pairs. None of this requires rebuilding an already compatible serving index, but the training targets still have to be produced. We separate three stages: selecting and preparing permitted data, encoding teacher targets, and optimizing or solving for the student.

Table 1. Component build costs. Nano’s time is measured; its dollar value prices that time at the recorded historical rental rate. Zero’s figures are approximate historical ledger estimates. Sources: `results/m15_e15_decision_audit.json`, `results/m13_build_record.json`, `results/m13_build_allocation.json`, `m7/LEDGER.md`.

Artifact	Fitting or optimization	Preparation and accounting boundary
Closed-form probe	4-7 minutes per registered configuration on an A100 for fitting and development scoring, including fit-query encoding except for cached Stella targets	coarse intervals between output files; downloads, data assembly, and index construction excluded
Zero	approximately 20 minutes of retraining on an RTX 3080, historical estimate	changing teachers was estimated to require 8-12 hours of target re-encoding; not a measured complete rebuild
Nano	57.3 A100 hours for 199,999,721 example presentations; about \$95 at the recorded \$1.664/hour rental rate	teacher targets already prepared; excludes data generation, preparation, recipe search, failed runs, evaluation, export, and engineering time

The Nano figure shows that the final student optimization did not require a cluster. It does not show that the entire project can be reproduced from scratch for \$95, or that this training dose is the cheapest way to

reach its quality. The table timings likewise show that fitting rows after teacher encoding can be short; they do not make the encoding stage free.

Zero uses project-approved sources including ESCI, FEVER-train, HotpotQA-train, Mr. TyDi English, and SQuAD-train, plus NQ-open and TriviaQA query text (m7/RECIPE.md). Nano uses a different pool and a 75% query / 25% document batch mix (m13/build_config.json). MS MARCO is validation-only in our distillation, and Nano’s pretrained bge-small backbone has its own MS MARCO exposure. Reusing an index does not resolve training-data permissions.

3.3 A process another team can follow

For a fixed index, first fit and evaluate the intended cheap query representation against its teacher’s document vectors. The closed-form probe in Section 4 is one inexpensive way to test a table recipe. If it meets the workload’s quality needs, it can itself be the candidate; if the plan is to train a different table or a transformer, evaluate that trained encoder separately. A probe is not a certificate for another recipe.

Freeze data splits and choices on representative development queries, retain the teacher as a reference, and evaluate on other queries. Record preparation and fitting separately. Finally, tune approximate search for the new query vectors rather than inheriting the teacher query path’s settings. The repository includes the probe driver (m15src/e8_towers.py), the served table recipe (m7/RECIPE.md), and Nano’s build configuration (m13/build_config.json). They expose the components and the provenance required to repeat the study; a standalone bring-your-own-teacher tool is not part of this release.

4. Teacher retrieval quality does not determine table quality

4.1 A controlled table comparison

We fit one closed-form table recipe to 10 checkpoints from six model families, all sharing a BERT WordPiece vocabulary. Ridge regression maps query token counts to the teacher’s query vector, anchored at the teacher’s embedding of each token. The fit list contains 337,981 queries screened against the held-out data. Each ridge weight was selected on CQADupStack physics and programmers, then frozen before the six public BEIR sets were scored (results/m15_e8_frozen_lambdas.json, results/m15_e8_towers.json, registered).

After observing that result, we added 17 checkpoints under the same recipe and freeze order: MiniLM, bge, e5, thenlper gte, arctic, Contriever, and TAS-B (results/m15_e8x_towers.json, exploratory). One arctic checkpoint failed the convergence gate and was excluded, leaving 26. Every teacher is evaluated over its own document vectors; this section compares teacher spaces, rather than swapping teachers into the Stella index.

4.2 The choice changes materially

Across 26 checkpoints, teacher quality and table quality have Spearman correlation +0.09 (95% interval -0.39 to +0.52). The interval allows moderate associations; the evidence does not establish independence. The concrete reversal is clearer: gte-large-en-v1.5 is the strongest registered teacher and yields the weakest registered table. Stella shares its backbone, tokenizer, and 1024-dimensional architecture but differs in training and readout, and yields the best table across all 26.

Table 2. Teacher-choice consequences under the closed-form recipe, six-set macro nDCG@10. Teacher and table scores use the same six datasets and each teacher’s own document index. Registered rows; the selection difference is an exploratory derivation in results/m15_e15_decision_audit.json.

Teacher	Teacher score	Table score	Table / teacher
gte-large-en-v1.5	0.5970	0.2455	0.411
Stella	0.5745	0.3974	0.692

The development screen chooses Stella. Selecting the strongest public teacher in hindsight would choose gte-large, so the screen’s selected table is higher by 0.1519 nDCG@10. This illustrates the cost of choosing

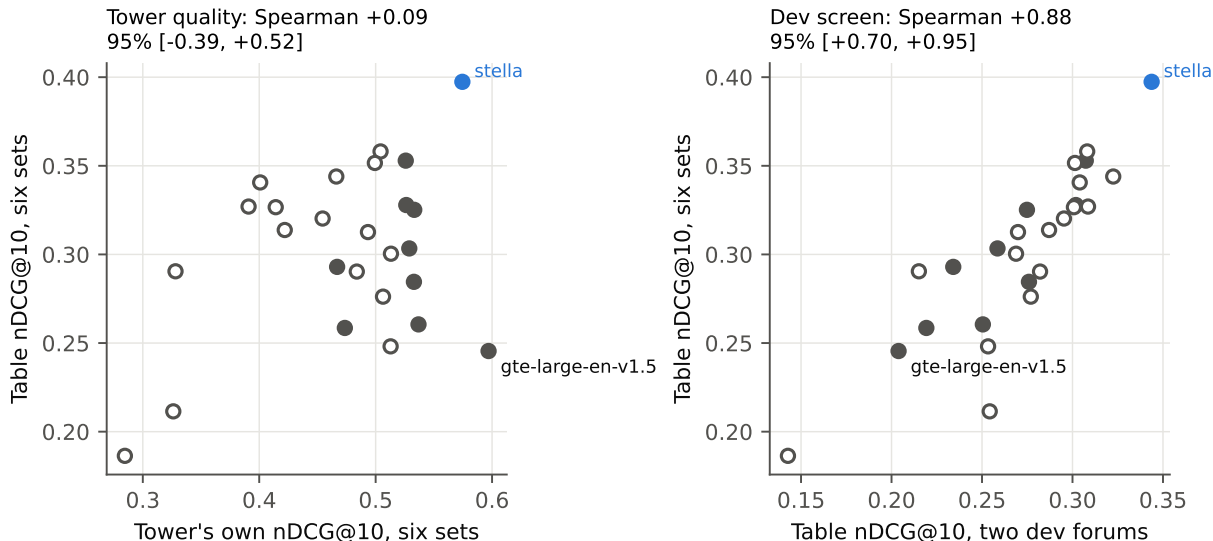


Figure 1: One closed-form recipe across 26 checkpoints. Filled points are the registered ten, hollow points the exploratory 16. Teacher quality weakly ranks table quality; directly evaluating the table on development queries ranks it much better. Intervals are 10,000-draw bootstraps over checkpoints.

by teacher quality even with the teacher’s public score available; it is not a prospectively tested model-card selection policy.

The table’s development score ranks its public score with Spearman 0.88 across the 26 (95% interval 0.70 to 0.95), against 0.90 across the registered ten. Resampling whole model families gives an interval of 0.74 to 0.95 (`results/m15_e14_screen_checks.json`, exploratory). Either development forum alone ranks the registered ten nearly as well (0.87 and 0.92). The result supports directly evaluating this cheap representation instead of inferring its quality from the teacher.

4.3 The screen has a target, not a universal winner

Stella wins the six-set table macro, but the development ranking agrees less strongly with the four sets without disclosed Stella overlap (Spearman 0.68). On those four, the selected table trails the best of the registered ten by 0.0048 and the best of the pooled 26 by 0.0278. Across the 26, it ranks fifth on SciFact and eleventh on TREC-COVID (`results/m15_e15_decision_audit.json`, exploratory). A screen’s development workload must reflect the quality objective; these forums do not select the best table for every domain.

Higher-dimensional checkpoints tend to produce weaker tables under this recipe. The association with absolute table quality is -0.44, while dimension correlates with teacher quality at +0.61. Its stronger association with retention, -0.76, partly reflects the teacher score in the denominator. Dimension is not parameter count and is confounded with training and readout; this is not a causal result about model size. The within-family observations and both absolute and relative correlations are recorded in the evidence file.

Vector fidelity is also an unreliable substitute for retrieval evaluation in these experiments. The e5 tables match their teacher query vectors more closely than Stella’s table does (mean cosine 0.90 and 0.89, against 0.78), yet retrieve worse. Three exploratory scalar diagnostics, including error relative to ranking margins, do not explain the teacher ranking reliably at $n = 10$ (`results/m15_e11_mechanism.json`, `results/m15_e11b_margin.json`). We have established a practical reversal and a useful direct screen, not its underlying cause.

The scope is one table recipe, one shared vocabulary, and six public text-retrieval datasets. Most teachers

trained on related source families; exposure for the registered ten is disclosed in `m15/e8_exposure.json`, while the exploratory 16 were not individually audited. An earlier project fit list had 1.31% held-out-query overlap; this comparison uses its cleaned successor. No result here demonstrates that the screen predicts the separately trained Zero recipe or transformer students.

5. What cheap encoding buys after retrieval

Table 3. BEIR-15 macro nDCG@10 under exact search over the Stella index, and query-encode latency (median over 100 real test queries of 5-12 words, batch one, four CPU threads, ONNX Runtime, Zero on NumPy, Apple M5 Pro; `results/m15_e1_latency.json`). Retention is a ratio of macros. Registered.

Query side	nDCG@10	Retention	Encode p50
Stella query path	0.5614	1.000	31.6 ms
Nano	0.5081	0.905	2.25 ms
Nano + BM25 (DBSF@100)	0.5110	0.910	2.25 ms + BM25
Zero	0.4572	0.814	0.044 ms
Zero + BM25 (DBSF@100)	0.4933	0.879	0.044 ms + BM25
BM25 alone	0.4006	0.713	

Nano keeps nine tenths of Stella’s quality at one fourteenth of its encode latency; Zero keeps four fifths at one seven-hundredth and beats BM25 on 11 of 15 datasets. Nano is a mid-strength small query encoder: on BEIR-15, it scores below bge-small on bge-small’s own index (0.5171) and LEAF on its arctic-embed-m index (0.5402, at 1.39 ms). Other distilled query encoders report 92.5% to 97.7% against their own teachers, with different datasets, protocols, and retention definitions; these percentages are not a controlled ranking. Nano shares the Stella index with Zero and Stella, enabling Appendix C’s routing and blending; its 34.5M student against a 400M teacher is also a wider size gap than LEAF’s 23M against 109M. Retention varies more across datasets than tiers, from 0.667 (Zero on TREC-COVID) to 0.988 (Nano on Quora). Figure 4 in Appendix E shows the encoding frontier; the evidence file gives the full per-dataset spread.

A table’s query needs a wider graph search. We served each tier from Qdrant (v1.19.1, native macOS build) over a one-million-passage MS MARCO subset that keeps every judged positive and fills the rest at random (a diagnostic, not full MS MARCO; MS MARCO is used for validation only), and over FiQA (57,638 documents). For each tier, we swept HNSW `ef` and three quantizations (int8 scalar, 1-bit binary, and 4-bit TurboQuant, each with rescoring and 1x-4x oversampling). We selected the cheapest end-to-end setting within 1%, 2%, and 5% of that tier’s own exact nDCG@10 (registered method). Every segment was HNSW-indexed before timing (`indexing_threshold` set to 1 KB instead of the 10,000 KB default, so no vector was served by a plain scan); this is a benchmark setting, not a recommendation. At the same unquantized setting on the 1M collection (`ef` 16), approximate search costs Zero 16.3% of its exact score, Nano 7.0%, and Stella 4.4%. Their top-10 lists agree with exact search on 82%, 91%, and 93% of neighbors (`results/m15_e2_ann_msarco1m.json`). Median cosine to the nearest document is 0.60 for Zero, 0.74 for Nano, and 0.78 for Stella. The table’s averaged query vector sits farther from documents used to build the graph, consistent with, though not a test of, a harder graph search.

The saving survives, and shrinks. Within 1% of exact score on the 1M collection, Zero needs `ef` 512 with binary quantization and 2x oversampling (search p50 1.09 ms), Nano `ef` 256 with 4x oversampling (0.87 ms), and Stella `ef` 64 with TurboQuant (0.66 ms). Including encoding, that is 1.11 ms for Zero, 2.48 ms for Nano, and 21.3 ms for Stella. Nano costs 2.2 times Zero at the 1% target and 3.1 times at 5%, down from 62 times for encoding alone on these short queries. These targets preserve each tier’s own exact quality; they do not compare systems at equal absolute nDCG@10. FiQA shows the same ordering (at `ef` 16: 6.8%, 3.2%, and 1.5% lost); Nano costs 3.8 to 4.3 times Zero end to end (`results/m15_e2_ann_fiqa.json`). Absolute laptop latencies move with background load: an earlier FiQA run with another job using the CPU measured Nano’s encode at 1.75 ms against 2.83 ms in the rerun, while the ratio stayed within 3.4 to 4.3 (`results/m15_e2_ann_fiqa_run1.json`). We therefore read absolute latencies from E1 and the MS MARCO run.

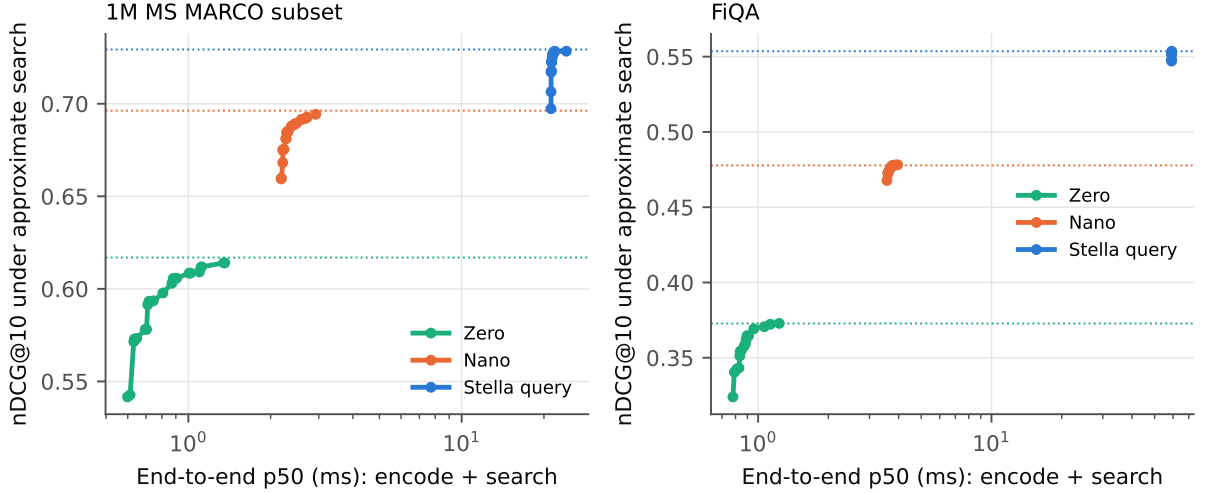


Figure 2: nDCG@10 under approximate search against end-to-end p50 (encode plus search); each point is the fastest setting reaching that quality, and dotted lines mark each tier’s exact score. Left: 1M MS MARCO subset; right: FiQA.

The engineering implication is to measure search again after a query encoder changes, even when the document vectors and graph are unchanged. The table’s exact quality loss and its approximate-search loss are distinct. The speedup remaining after retuning is still useful on the measured workload, but it is much smaller than its encoding speedup.

6. Evaluation boundaries and limitations

The exact quality rows use 15 BEIR datasets (`results/m20_beir15_run.json`). Four datasets were held out for one registered test: FEVER, DBpedia-entity, and CQADupStack android and english. Their published aggregate results enter BEIR-15 but play no fitting or diagnostic role. Ridge weights, router thresholds, and blend weights use the other two forums. The trained tiers were built earlier on the project’s development suite.

The teacher, Zero, and Nano are not matched-data architecture ablations. Zero trained on FEVER-train and HotpotQA; Nano’s pool excluded FEVER. Zero’s higher scores on FEVER-family datasets therefore mix architecture with data. Our single-prompt Stella path scores 0.5614 on BEIR-15, below its model card’s 58.97 under task-specific instructions, a 400-token limit, and bf16. These results characterize our pinned serving path. Appendix B preserves the registered comparator contrasts and all four held-out aggregates, including unresolved and negative outcomes.

We trained the served recipes against one teacher. The broader teacher comparison tests closed-form tables only, with 16 of 26 checkpoints added after the registered observation. Neither teacher-rank correlations nor checkpoint bootstraps establish a general law of distillation. BEIR-15 and the new cost and selection audit are descriptive; query-resampling intervals exclude training-seed variation.

Deployment measurements use one laptop CPU, one runtime, and two collections. We did not measure server throughput or GPU serving. The MS MARCO subset retains judged positives while removing most competing passages, so its absolute nDCG@10 is optimistic. Its search penalties concern that subset. End-to-end latency ratios preserve each tier’s own quality and do not establish equal-quality superiority. Build costs price recorded compute and estimates; they omit the complete research and engineering effort.

A symmetric small model is also a real alternative. Nano scores below bge-small and LEAF on their own indexes in our BEIR-15 comparison. Its practical value depends on preserving an existing Stella index and

offering compatible query paths. We do not measure the full cost of migrating to another index, so index reuse is a deployment constraint and capability, not a quantified savings claim.

7. Conclusion

Cheap query encoders can reuse a strong document index, but teacher quality, fitting cost, and serving cost answer different questions. Under one closed-form table recipe, the strongest teacher yields the weakest registered table; a development ranking of the tables transfers much better than the teacher ranking, with meaningful task sensitivity. Training the served transformer on cached targets costs about \$95 in recorded rental time, not \$95 for the complete project. The table’s encoding speedup survives approximate retrieval but falls from about 60-fold to two- to three-fold over Nano on our larger diagnostic. For a team considering this design, the evidence supports evaluating the intended cheap representation, budgeting target preparation separately, and retuning search for its vectors before deciding whether the swap is worthwhile.

Appendix A. What a mean-pooled table absorbs

The full proof is in `m15/PROOF_ABSORB.md`. Let a table with rows w_t pool a query with weights $\alpha_t(q) \geq 0$ that sum to one (mean pooling, and Zero’s count-saturated pooling), then normalize. (1) Applying $x \mapsto Ax + b$ after pooling equals pooling the table $w'_t = Aw_t + b$, because $\sum_t \alpha_t(Aw_t + b) = A \sum_t \alpha_t w_t + b$; under sum pooling the offset would scale with query length instead. (2) Reweighting tokens by fixed $g_t > 0$ and renormalizing equals the table $g_t w_t$ up to a positive scalar that the final normalization removes. Centering, whitening, principal-component removal, IDF or SIF weighting and their compositions are therefore tables of the same shape. The lemmas do not cover nonlinear maps, pooling weights that depend on count patterns, or word order.

Appendix B. Registered evaluations

Contrast	Partition	Difference in nDCG@10	Registered outcome
Nano - bge-small	clean-4	+0.017648 (one-sided lower bound +0.003674)	established
Nano - bge-small	all six	+0.027449	established
Nano - LEAF	all six	+0.016181	established
Nano - LEAF	clean-4	-0.001063	unresolved
Nano - bge-small	held-out NDO-3, dataset-weighted	+0.0032 [-0.0069, +0.0134]	registered descriptive
Nano - bge-small	held-out NDO-3, query-pooled	-0.0121 [-0.0219, -0.0024]	registered descriptive
Nano - LEAF	held-out NDO-3	-0.0389 [-0.0488, -0.0290]	registered descriptive

Clean-4 is NFCorpus, SCIDOCs, SciFact and TREC-COVID, the six-set members Stella’s recorded training data does not name. NDO-3 weights DBpedia-entity 0.5 and the two held-out CQADupStack forums 0.25 each; FEVER’s row is in `m15/EVIDENCE.md`. “Unresolved” means the registered rule did not establish a difference; no equivalence interval was computed. Sources: `m21/BENCHMARKS.md`, `results/m13_reserved_run.json`.

The originally held-out four, copied only from the published aggregate receipt `results/m13_reserved_run.json`:

System	FEVER	DBpedia	CQADup android	CQADup english
nano-dense	0.6231	0.4190	0.4774	0.4298
zero-dense	0.6978	0.3900	0.4431	0.3690
stella-query	0.8207	0.4603	0.5275	0.5183

System	FEVER	DBpedia	CQADup android	CQADup english
bge-small-en-v1.5	0.8662	0.4003	0.4761	0.4558
leaf-ir-asym	0.8671	0.4520	0.5260	0.4707
bm25	0.5036	0.3045	0.3952	0.3481
zero+bm25 dbsf@100	0.7559	0.4030	0.4629	0.4016
nano+bm25 dbsf@100	0.6879	0.4296	0.4750	0.4323

Zero’s original registered dense test missed the LightRetriever-derived bar. Its comparison with BM25 failed the Holm threshold despite a positive unadjusted interval; its fused comparison with the OpenSearch reference was unresolved (`m21/BENCHMARKS.md`). These outcomes remain part of the research record and establish neither superiority nor equivalence. Full six-set and BEIR-15 rows for every system are in `m15/EVIDENCE.md`.

Appendix C. Routing and vector blending

C.1 Routing headroom

On the 12 BEIR-15 datasets outside the held-out test, averaging per-dataset shares, Zero and Nano both score 0 on 17% of queries, tie above 0 on 26%, and Zero is higher on 18% and Nano on 39% (pooling all queries gives 56% ties, because Quora is large and tied on 72% of its queries). A per-query oracle with relevance labels picks the better tier and reaches 0.5473 macro nDCG@10, against 0.5161 for always-Nano and 0.5580 for Stella. Sending only 15% of each dataset’s queries to Nano, largest gains first, already matches always-Nano; at 25% it reaches 0.5334 (`results/m15_e5_oracle.json`, registered). Without Climate-FEVER, the one FEVER-family set among the 12, always-Nano scores 0.5406 and the oracle needs 20% of queries to match it (0.5453). This label-aware oracle bounds what routing can gain.

C.2 Frozen routers

A router must decide before running Nano. The signals are subwords per word, pooled-vector norm before normalization, word count, the gap between Zero’s first and tenth document scores, and the number of documents shared by Zero and BM25 in their top ten. For each signal, we fitted its direction and threshold on the two development forums at Nano budgets of 10%, 25%, and 50%, then froze them. We compared each router with random routing at its realized Nano share and with the oracle at that share (`results/m15_e6_router.json`, `results/m15_e10_router.json`, registered).

Signal	Available after	Gain over random, 25% budget	Oracle gain share	Sets
Fertility	encoding	+0.0049	0.10	12
Pooled vector norm	encoding	+0.0016	0.03	12
Word count	encoding	+0.0018	0.04	12
Zero score margin	Zero search	+0.0175	0.25	6
Zero/BM25 overlap	both searches	+0.0154	0.31	6

The three query-only signals carry little. Zero’s retrieval margin carries more: sending queries with the narrowest margin to Nano (28% of each dataset’s queries on average, 24% pooled) scores 0.4865 on the six sets, against 0.4690 for random routing at that share and 0.5317 for always-Nano. Gains are largest where Zero is weakest: +0.043 on SciFact and +0.032 on TREC-COVID. Sending routed queries to the Appendix C.3 blend instead of Nano scores 0.4834, 0.0031 lower. The agreement signal counts shared documents; at the 10% budget its frozen threshold routes no queries.

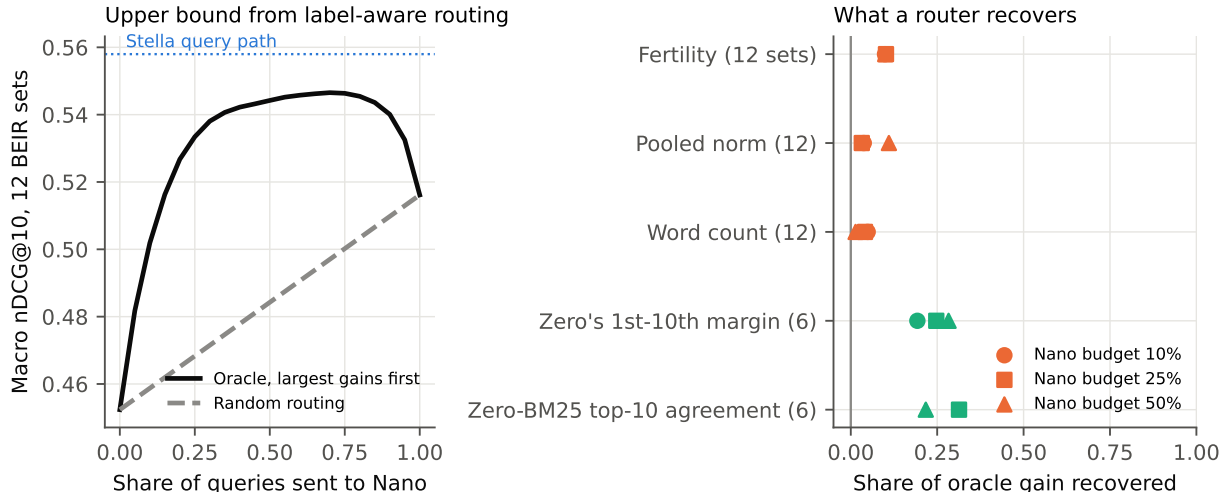


Figure 3: Left: the upper bound from label-aware routing on the 12 evaluation sets, against random routing. Right: the share of the oracle’s gain over random routing each frozen router recovers, per Nano budget; orange signals are known before the search, green ones after Zero’s own search.

C.3 Cost and vector blending

A margin router first searches with Zero, then encodes and searches again for escalated queries. Its cost therefore includes two searches on those queries. Section 5’s modest end-to-end gap limits the latency it can save on this workload; the oracle is a relevance upper bound, not a deployable speedup.

Zero and Nano write into one space, so a system can search once with $\text{normalize}((1 - a) \text{Nano} + a \text{Zero})$, adding 0.044 ms to Nano. The development forums chose $a = 0.3$, raising Nano there from 0.4247 to 0.4295. On the six public sets, the frozen blend lowered it by 0.0051 (95% interval -0.0092 to -0.0011), gaining only on ArguAna (+0.005) and losing most on TREC-COVID (-0.016). For Stella, the forums chose no blend (`results/m15_e9_blend.json`, registered descriptive; a pre-freeze code check printed SciFact’s curve, recorded in `m15/MEASUREMENTS.md`). The forums that ranked towers in Section 4 did not select a useful blend weight.

Appendix D. Query diagnostics

D.1 Shuffle sensitivity

The unigram table is invariant to permutations of its token counts. On 3,727 queries of six public sets, the Stella-minus-Zero gap correlates with Stella’s own nDCG@10 drop under five word shuffles (Spearman 0.52). The 40% of queries on which shuffling hurts Stella carry about 90% of the observed net gap (`results/m15_e12_failure_modes.json`, exploratory). This localizes a failure pattern but does not identify its cause: both differences share Stella’s full-query score, shuffling changes more than word order, and positive and negative query gaps cancel in the net total.

Stratifying by broad bins of Stella’s score leaves the association at 0.53. A separate control perturbs Stella’s query vector once in an isotropic random direction at the same cosine distance as one shuffle. Among 3,069 queries with positive Stella scores, that perturbation costs 0.009 nDCG@10 on average, against 0.047 for shuffling (`results/m15_e12b_fragility.json`, exploratory). Broad score bins do not remove all shared-score coupling, and equal-distance isotropic noise does not match a language perturbation’s direction relative to ranking boundaries. These controls establish a bounded diagnostic, not that word order explains a particular fraction of the loss or that general fragility has been excluded.

Subwords per word correlate with the gap at 0.10. That feature measures fragmentation, not token rarity or

training support. Two selected FiQA failures illustrate the diagnostic (`results/m15_e13_robustness.json`, exploratory). For “What credit card information are offline US merchants allowed to collect for purposes other than the transaction?”, the first relevant answer ranks first with Stella, 59th after shuffling, 11th with Nano, and outside Zero’s top 100. For “What can I do with a physical stock certificate for a now-mutual company?”, the corresponding ranks are 1, 8, 13, and 29. These queries were selected for large shuffle sensitivity and large Stella-minus-Zero gaps, so they do not represent the typical query.

D.2 Short and unfinished queries

We cut each SciFact, NFCorpus, and FiQA test query and measured the share of its own full-query nDCG@10 each tier keeps (macro over the three sets, registered, `results/m15_e4_prefix.json`):

Cut	Stella query	Nano	Zero	BM25
First word	0.293	0.288	0.297	0.362
First two words	0.389	0.381	0.387	0.448
First three words	0.479	0.468	0.461	0.528
First half of the words	0.610	0.597	0.604	0.656
First half of the characters	0.549	0.533	0.547	0.488

The dense tiers’ shares differ by at most 0.02 at every cut shown and by 0.034 at 75% of the words (0.846, 0.829 and 0.812 for Stella, Nano and Zero). This is a descriptive similarity on cut test queries, not an equivalence test or real typing. BM25 keeps a larger share on word prefixes and a smaller one when a cut lands inside a word.

Appendix E. Quality spread and serving compatibility

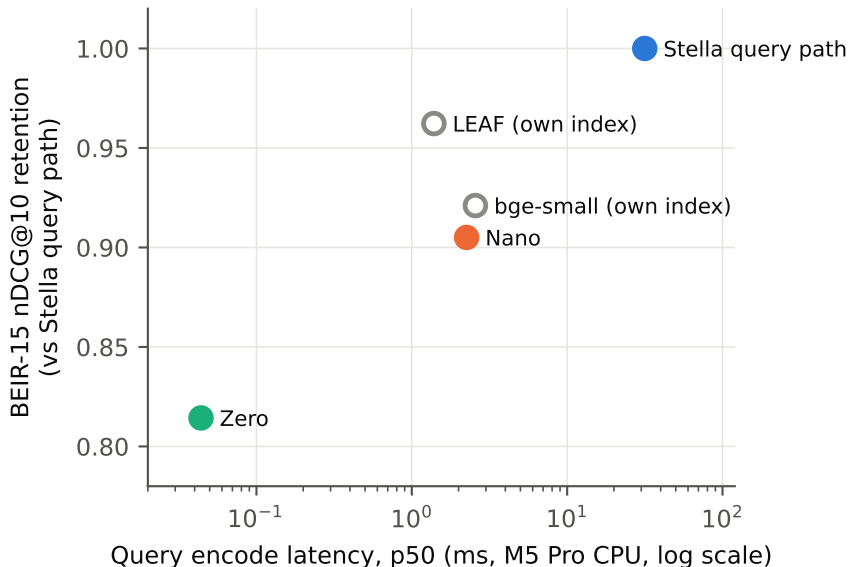


Figure 4: BEIR-15 retention against encode latency. Hollow markers are reference systems on their own indexes (bge-small 0.5171 at 2.57 ms, LEAF 0.5402 at 1.39 ms).

Per-dataset BEIR-15 rows for all eight systems, E8’s per-dataset tables and the full E1 latency inventory are in `m15/EVIDENCE.md`.

A swap is legal when the replacement emits vectors aligned with the index’s document space, with the same width, normalization and similarity; our served paths match their references to 4.5e-08 (Zero), 1.2e-07

(Nano) and a minimum cosine of 1.00000000 (Stella through the published document graph). Three things fail silently: Stella without its prompt scores at cosine 0.80 against the correct vector; Stella’s tokenizer pads to 512 by default, which drops cosine to 0.35; and FastEmbed’s mean pooling returns float64 for mean-pooled models, Nano among them, which doubles memory and changes the raw bytes (qdrant/fastembed issue #752, recorded in the project audit; our measurements pool in float32 with ONNX Runtime directly).

Appendix F. Reproducibility

Every figure regenerates from committed JSON (`python m15/figures/make_figures.py`) and every number in `m15/EVIDENCE.md` from `python m15/make_evidence.py`. M15 result files (`results/m15/*.json`), except E8’s frozen-lambda file (written on the pod and bound by hash into E8’s result), carry the script hash, git commit, seed and machine; model and dataset revisions are recorded in the result, in the scripts it names, or in the result it was derived from (E8’s frozen lambdas are bound by hash into E8’s result; E12 and E12b take their pins from the committed M20 rows). Earlier results carry the provenance records of the milestone that produced them. Methods and reviews: `m15/MEASUREMENTS.md`, `m15/REVIEWS/`.

E15 derives teacher-choice consequences and component build costs from five published JSON receipts and the Zero ledger, without new training or raw evaluation access. To check E15 without overwriting its committed receipt, write a new result to an unused scratch path:

```
.venv/bin/python m15src/e15_decision_audit.py \
  --output work/m15/e15-check/result.json
```

The script records input and code hashes; the paper distinguishes measured training time, priced component cost, and historical estimates.

References

Primary-source checks and supporting passages are recorded in `m15/RELATED_WORK.md` and `m15/REVIEWS/2026-10-01-owner-literature.md`.

- [QED] Yuxuan Wang and Hong Lyu. Query Encoder Distillation via Embedding Alignment is a Strong Baseline Method to Boost Dense Retriever Online Efficiency. SustainNLP Workshop at ACL, 2023. [arXiv:2306.11550](#).
- [EmbedDistill] Seungyeon Kim, Ankit Singh Rawat, Manzil Zaheer, et al. EmbedDistill: A Geometric Knowledge Distillation for Information Retrieval. 2023. [arXiv:2301.12005](#).
- [LEAF] Robin Vujanic and Thomas Rueckstiess. LEAF: Knowledge Distillation of Text Embedding Models with Teacher-Aligned Representations. 2025, revised 2026. [arXiv:2509.12539](#).
- [NanoVDR] Zhuchenyang Liu, Yao Zhang, and Yu Xiao. NanoVDR: Distilling a 2B Vision-Language Retriever into a 70M Text-Only Encoder for Visual Document Retrieval. 2026. [arXiv:2603.12824](#).
- [LightRetriever] Guangyuan Ma, Yongliang Ma, Xuanrui Gou, Zhenpeng Su, Ming Zhou, and Songlin Hu. LightRetriever: A LLM-based Text Retrieval Architecture with Extremely Faster Query Inference. ICLR, 2026. [arXiv:2505.12260](#).
- [pyNIFE] Stephan Tulkens. pyNIFE: Nearly Inference Free Embeddings in Python. Software, 2025. [Repository](#).
- [Model2Vec] MinishLab. Model2Vec: Fast State-of-the-Art Static Embeddings. Software. [Repository](#).
- [StaticEmb] Tom Aarsen. Train 400x faster Static Embedding Models with Sentence Transformers. Hugging Face, 15 January 2025. [Blog post](#).
- [Cho and Hariharan 2019] Jang Hyun Cho and Bharath Hariharan. On the Efficacy of Knowledge Distillation. ICCV, 2019. [Proceedings](#).
- [PROD] Zhenghao Lin, Yeyun Gong, Xiao Liu, et al. PROD: Progressive Distillation for Dense Retrieval. WWW, 2023. [arXiv:2209.13335](#).
- [Arabzadeh et al.] Negar Arabzadeh, Xinyi Yan, and Charles L. A. Clarke. Predicting Efficiency/Effectiveness Trade-offs for Dense vs. Sparse Retrieval Strategy Selection. CIKM, 2021. [arXiv:2109.10739](#).

- [QPP] Guglielmo Faggioli, Thibault Formal, Stefano Marchesin, Stéphane Clinchant, Nicola Ferro, and Benjamin Piwowarski. Query Performance Prediction for Neural IR: Are We There Yet? 2023. [arXiv:2302.09947](#).
- [BCT] Yantao Shen, Yuanjun Xiong, Wei Xia, and Stefano Soatto. Towards Backward-Compatible Representation Learning. CVPR, 2020. [Proceedings](#).
- [FCT] Vivek Ramanujan, Pavan Kumar Anasosalu Vasu, Ali Farhadi, Oncel Tuzel, and Hadi Pouransari. Forward Compatible Training for Large-Scale Embedding Retrieval Systems. CVPR, 2022. [arXiv:2112.02805](#).